Compiler Design Lab

Date: 13-2-2026 **Name:** S Vishwajith
Assignment No.: 7 **Register No.:** 23BCE1145

Aim:

To implement the SLR parser using C++.

Code:

```
#include <iostream>

#include <iomanip>

#include <vector>

#include <set>

#include <map>

using namespace std;


struct prod

{

   int lhsIdx;

   vector<pair<int, int>> rhsIdx; // nt/t, nt/tIdx - char

};

struct stackItem

{

   int type; // 0-state, 1-NT, 2-T

   int state;

   int ntIdx;

   int tIdx;

};
```

```
char startSym;
int startSymIdx;
set<char> terms, nterms;
vector<prod> prods;
vector<pair<int, int>> litems; // prodIdx, .pos - lr0 item
vector<vector<int>> lstates;   // stateNo - vector(litemsIdx)
vector<vector<int>> closures;  // ntIdx - vector(litemsIdx)
vector<int> cproc;
vector<int> fiproc, foproc;
vector<set<char>> first, follow;         // ntIdx - set(first) and set(follow)
map<pair<int, pair<int, int>>, int> gotos; // stateNo, (nt/t nt/tIdx) -> stateNo
vector<vector<string>> table;            // stateNo, t-ntIdx - Action
string input;
vector<stackItem> stack;

int findState(vector<int> v)
{
   set<int> state(v.begin(), v.end());
   for (int i = 0; i < lstates.size(); i++)
   {
      set<int> cstate(lstates[i].begin(), lstates[i].end());
      if (state == cstate)
         return i;
   }
   return -1;
}

int findLitem(pair<int, int> litem, vector<pair<int, int>> lritems)
```

```
{
   for (int i = 0; i < lritems.size(); i++)
   {
      if (litem == lritems[i])
         return i;
   }
   return -1;
}

int findInt(int i, vector<int> v)
{
   for (int j = 0; j < v.size(); j++)
   {
      if (i == v[j])
         return j;
   }
   return -1;
}

set<char> fillFirst(char t)
{
   int ntIdx = distance(nterms.begin(), nterms.find(t));
   if (fiproc[ntIdx])
      return first[ntIdx];
   fiproc[ntIdx] = 1;
   for (prod p : prods)
   {
      if (p.lhsIdx == ntIdx)
```

```
        {
            int fnt = p.rhsIdx[0].first;
            int fIdx = p.rhsIdx[0].second;
            if (fnt)
                first[ntIdx].insert(*next(terms.begin(), fIdx));
            else
            {
                char tgt = *next(nterms.begin(), fIdx);
                for (char c : fillFirst(tgt))
                    first[ntIdx].insert(c);
            }
        }
    }
    return first[ntIdx];
}

set<char> fillFollow(char t)
{
    int ntIdx = distance(nterms.begin(), nterms.find(t));
    if (foproc[ntIdx])
        return follow[ntIdx];
    foproc[ntIdx] = 1;
    for (prod p : prods)
    {
        for (int i = 0; i < p.rhsIdx.size(); i++)
        {
            int fnt = p.rhsIdx[i].first;
            int fntIdx = p.rhsIdx[i].second;
```

```
        if (fnt == 0 && fntIdx == ntIdx)
        {
            if (i == p.rhsIdx.size() - 1)
            {
                char tgt = *next(nterms.begin(), p.lhsIdx);
                for (char c : fillFollow(tgt))
                    follow[ntIdx].insert(c);
            }
            else
            {
                int fn = p.rhsIdx[i + 1].first;
                int fnIdx = p.rhsIdx[i + 1].second;
                if (fn)
                    follow[ntIdx].insert(*next(terms.begin(), fnIdx));
                else
                {
                    char tgt = *next(nterms.begin(), fnIdx);
                    for (char c : fillFirst(tgt))
                        follow[ntIdx].insert(c);
                }
            }
        }
    }
  }

  return follow[ntIdx];
}
```

```
vector<int> findClosure(int nt)
{
   if (cproc[nt])
      return closures[nt];
   cproc[nt] = 1;
   vector<int> litems;
   for (int i = 0; i < prods.size(); i++)
   {
      if (prods[i].lhsIdx == nt)
      {
         litems.push_back(findLitem(pair<int, int>(i, 0), ::litems));
         if (prods[i].rhsIdx[0].first == 0 &&
            prods[i].rhsIdx[0].second != nt)
         {
            vector<int> vc = findClosure(prods[i].rhsIdx[0].second);
            for (int i : vc)
            {
               if (findInt(i, litems) == -1)
                  litems.push_back(i);
            }
         }
      }
   }
   closures[nt] = litems;
   return litems;
}

void constructClosures()
```

```
{
   for (int i = 0; i < prods.size(); i++)
      litems.push_back(pair<int, int>(i, 0));
   cproc = vector<int>(nterms.size(), 0);
   closures = vector<vector<int>>(nterms.size());
   for (int i = 0; i < nterms.size(); i++)
      findClosure(i);
}

void printProd(prod pro, int i)
{
   char l = *next(nterms.begin(), pro.lhsIdx);
   if (l == 'Z')
      cout << startSym << "'" << "->";
   else
      cout << l << "->";
   int x = 0;
   for (pair<int, int> ridx : pro.rhsIdx)
   {
      char c;
      if (ridx.first == 0)
         c = *next(nterms.begin(), ridx.second);
      else
         c = *next(terms.begin(), ridx.second);
      if (x++ == i)
         cout << ".";
      cout << c;
   }
```

```
    if (i == pro.rhsIdx.size())
        cout << ".";
    cout << endl;
}

void findGoto(int lstateNo, char c)
{

    int nt = 1;
    if (nterms.count(c))
        nt = 0;
    int ntIdx;
    if (nt == 0)
        ntIdx = distance(nterms.begin(), nterms.find(c));
    else
        ntIdx = distance(terms.begin(), terms.find(c));
    vector<int> lritems;
    int found = 0;
    for (int x : lstates[lstateNo])
    {
        pair<int, int> p = litems[x];
        prod pr = prods[p.first];
        int pos = p.second;
        if (pr.rhsIdx.size() > pos)
        {
            if (pair<int, int>(nt, ntIdx) == pr.rhsIdx[pos])
            {
```

```
            found = 1;
            pair<int, int> newitem(p.first, pos + 1);
            int newIdx = findLitem(newitem, litems);
            if (newIdx == -1)
            {
                litems.push_back(newitem);
                lritems.push_back(litems.size() - 1);
            }
            else
                lritems.push_back(newIdx);
            if (pos + 1 < pr.rhsIdx.size())
            {
                pair<int, int> clositem = pr.rhsIdx[pos + 1];
                if (clositem.first == 0)
                {
                    vector<int> itemclos = closures[clositem.second];
                    for (int i : itemclos)
                    {
                        if (findInt(i, lritems) == -1)
                            lritems.push_back(i);
                    }
                }
            }
        }
    }
}
pair<int, int> nti(nt, ntIdx);
pair<int, pair<int, int>> gt(lstateNo, nti);
```

```
    int gtstatenum = -1;
    if (!found)
    {
        pair<pair<int, pair<int, int>>, int> val(gt, gtstatenum);
        gotos.insert(val);
    }
    else if (findState(lritems) == -1)
    {
        lstates.push_back(lritems);
        gtstatenum = lstates.size() - 1;
    }
    else
        gtstatenum = findState(lritems);
    pair<pair<int, pair<int, int>>, int> val(gt, gtstatenum);
    gotos.insert(val);
}

void printGotos()
{
    int x = 0;
    int s = nterms.size() + terms.size() - 1;
    for (pair<pair<int, pair<int, int>>, int> p : gotos)
    {
        int nt = p.first.second.first;
        int ntIdx = p.first.second.second;
        char ntc;
        if (nt)
            ntc = *next(terms.begin(), ntIdx);
```

```
        else
            ntc = *next(nterms.begin(), ntIdx);
        int gst = p.second;
        if (gst != -1)
            cout << "goto(" << p.first.first << "," << ntc << "): " << gst << " | ";
        else
            cout << "goto(" << p.first.first << "," << ntc << "): - | ";
        x = (x + 1) % s;
        if (x == 0)
            cout << endl;
    }
}

void printClosures()
{
    for (int i = 0; i < closures.size(); i++)
    {
        cout << *next(nterms.begin(), i) << ": " << endl;
        for (int p : closures[i])
        {
            pair<int, int> pr = *next(litems.begin(), p);
            prod pro = prods[pr.first];
            printProd(pro, pr.second);
        }
        cout << endl;
    }
}
```

```
void printStates()
{
    for (int j = 0; j < lstates.size(); j++)
    {
        cout << "I" << j << endl;
        for (int p : lstates[j])
        {
            pair<int, int> pr = *next(litems.begin(), p);
            prod pro = prods[pr.first];
            printProd(pro, pr.second);
        }
        cout << endl;
    }
}

void printFollows()
{
    for (int i = 0; i < follow.size(); i++)
    {
        char c = *next(nterms.begin(), i);
        cout << c << ": {";
        for (char x : follow[i])
            cout << x << " ";
        cout << "}" << endl;
    }
    cout << endl;
}
```

```
void fillTable()
{
    for (pair<pair<int, pair<int, int>>, int> p : gotos)
    {
        int gst = p.second;
        int sn = p.first.first;
        int nt = p.first.second.first;
        int ntIdx = p.first.second.second;
        if (gst == -1)
            continue;
        if (nt)
            table[sn][ntIdx] = "s" + to_string(gst);
        else if (startSymIdx != ntIdx)
            table[sn][ntIdx + terms.size()] = to_string(gst);
    }
    for (int i = 0; i < lstates.size(); i++)
    {
        for (int lritem : lstates[i])
        {
            int pos = litems[lritem].second;
            int prodIdx = litems[lritem].first;
            if (pos == prods[prodIdx].rhsIdx.size())
            {
                for (char c : terms)
                {
                    int cIdx = distance(terms.begin(), terms.find(c));
                    if (follow[prods[prodIdx].lhsIdx].count(c))
                    {
```

```
                string fstring;
                if (prodIdx == 0)
                    fstring = "ACC";
                else
                    fstring = "r" + to_string(prodIdx);
                if (table[i][cIdx] == "-")
                    table[i][cIdx] = fstring;
            }
        }
    }
  }
}

void printTable()
{
  cout << "SLR Parsing Table: " << endl;
  int nnt = nterms.size() - 1;
  int ntt = terms.size();
  cout << setw(5) << "STATE"
      << setw(4) << "|"
      << setw(ntt * 5) << "ACTION"
      << setw(ntt * 5) << "|"
      << setw(nnt * 2) << "GOTO"
      << setw(nnt * 2) << "|" << endl;
  cout << setw(5) << "-"
      << setw(4) << "|";
  for (char c : terms)
```

```
        cout << setw(5) << c << setw(5) << "|";
    for (char c : nterms)
    {
        if (c != 'Z')
            cout << setw(2) << c << setw(2) << "|";
    }
    cout << endl;
    for (int i = 0; i < table.size(); i++)
    {
        cout << setw(5) << i << setw(4) << "|";
        int j = 0;
        for (; j < terms.size(); j++)
            cout << setw(5) << table[i][j] << setw(5) << "|";
        for (; j < table[i].size() - 1; j++)
            cout << setw(2) << table[i][j] << setw(2) << "|";
        cout << endl;
    }
}

void printStack()
{
    string out;
    for (stackItem s : stack)
    {
        if (s.type == 0)
            out += to_string(s.state);
        else if (s.type == 1)
            out += *next(nterms.begin(), s.ntIdx);
```

```
      else if (s.type == 2)
         out += *next(terms.begin(), s.tIdx);
   }
   cout << setw(10) << out << setw(10) << "|";
}

void parse(string in)
{
   in = in + '$';
   int i = 0;
   int j = 0;
   int acc = 0;
   stackItem init;
   init.type = 0;
   init.state = 0;
   stack.push_back(init);
   cout << setw(10) << "STACK"
      << setw(10) << "|"
      << setw(10) << "INPUT"
      << setw(10) << "|"
      << setw(20) << "ACTION"
      << setw(20) << "|" << endl;
   while (i < in.size() && acc == 0)
   {
      printStack();
      cout << setw(10) << in.substr(i) << setw(10) << "|";
      int tIdx = distance(terms.begin(), terms.find(in[i]));
      string a = table[stack[j].state][tIdx];
```

```
if (a[0] == 's')
{
   j += 2;
   stackItem newItem;
   newItem.type = 2;
   newItem.tIdx = distance(terms.begin(), terms.find(in[i++]));
   stack.push_back(newItem);
   newItem.type = 0;
   newItem.state = stoi(a.substr(1));
   newItem.tIdx = -1;
   stack.push_back(newItem);
   cout << setw(20) << "SHIFT " + to_string(newItem.state)
       << setw(20) << "|" << endl;
}
else if (a[0] == 'A')
{
   cout << setw(20) << "ACCEPT"
       << setw(20) << "|" << endl;
   exit(0);
}
else if (a[0] == 'r')
{
   int prodIdx = stoi(a.substr(1));
   prod rprod = prods[prodIdx];
   vector<pair<int, int>> rIdx;
   int ntop = j - 2 * rprod.rhsIdx.size() + 1;
   if (ntop < 0)
   {
```

```
    cout << setw(20) << "ERROR"
        << setw(20) << "|" << endl;
    exit(0);
}
for (int x = ntop; x <= j; x += 2)
{
    stackItem s1 = stack[x];
    rIdx.push_back(
        pair<int, int>(
            s1.type - 1,
            s1.type == 1 ? s1.ntIdx : s1.tIdx));
}
while (j >= ntop)
{
    j--;
    stack.pop_back();
}
if (rIdx == rprod.rhsIdx)
{
    stackItem s2;
    s2.type = 1;
    s2.ntIdx = rprod.lhsIdx;
    int gt = stoi(
        table[stack[j].state][terms.size() + s2.ntIdx]);
    stackItem s3;
    s3.type = 0;
    s3.state = gt;
    stack.push_back(s2);
```

```
    stack.push_back(s3);
    j += 2;
    char l = *next(nterms.begin(), rprod.lhsIdx);
    string nout = string(1, l) + "->";
    for (pair<int, int> ridx : rprod.rhsIdx)
    {
        char c;
        if (ridx.first == 0)
            c = *next(nterms.begin(), ridx.second);
        else
            c = *next(terms.begin(), ridx.second);
        nout += c;
    }
    cout << setw(20) << "REDUCE " + nout
         << setw(20) << "|" << endl;
}
else
{
    cout << setw(30) << "REDUCE FAILED"
         << setw(10) << "|" << endl;
    cout << "Stack RHS: ";
    for (pair<int, int> p : rIdx)
    {
        char c;
        if (p.first == 0)
            c = *next(nterms.begin(), p.second);
        else
            c = *next(terms.begin(), p.second);
```

```
            cout << c << " ";
        }
        cout << endl
            << "Target RHS: ";
        for (pair<int, int> p : rprod.rhsIdx)
        {
            char c;
            if (p.first == 0)
                c = *next(nterms.begin(), p.second);
            else
                c = *next(terms.begin(), p.second);
            cout << c << " ";
        }
        exit(0);
      }
    }
    else
    {
      cout << setw(20) << "ERROR"
          << setw(20) << "|" << endl;
      exit(0);
    }
  }
}

int main()
{
  int stlr = 0;
```

```
string in, t;
vector<pair<int, int>> v;
vector<prod> tempProds;
prod p;
cout << "Enter terminals seperated by comma: ";
cin >> in;
for (int i = 0; i < in.size(); i += 2)
   terms.insert(in[i]);
cout << "Enter non-terminals seperated by comma: ";
cin >> in;
for (int i = 0; i < in.size(); i += 2)
   nterms.insert(in[i]);
terms.insert('$');
cout << "Enter starting symbol: ";
cin >> startSym;
cout << "Enter productions seperated by |: ";
nterms.insert('Z');
startSymIdx = distance(nterms.begin(), nterms.find('Z'));
cin >> in;
for (int i = 0; i < in.size(); i++)
{
   char c = in[i];
   if (c == '|')
   {
      p.rhsIdx = v;
      tempProds.push_back(p);
      p.rhsIdx.clear();
      v.clear();
```

```
        stlr = 0;
    }
    else if (c == '-')
    {
        stlr = 1;
        i++;
    }
    else if (stlr == 0)
        p.lhsIdx = distance(nterms.begin(), nterms.find(c));
    else if (terms.count(c))
        v.push_back(
            pair<int, int>(
                1,
                distance(terms.begin(), terms.find(c))));
    else if (nterms.count(c))
        v.push_back(
            pair<int, int>(
                0,
                distance(nterms.begin(), nterms.find(c))));
    else
        cout << "ERROR";
}
cout << "Enter input string: ";
cin >> input;
p.rhsIdx = v;
tempProds.push_back(p);
p.rhsIdx.clear();
v.clear();
```

```
p.lhsIdx = distance(nterms.begin(), nterms.find('Z'));
v.push_back(
   pair<int, int>(
      0,
      distance(nterms.begin(), nterms.find(startSym))));
p.rhsIdx = v;
prods.push_back(p);
for (prod pr : tempProds)
   prods.push_back(pr);
cout << endl
    << "Augmented productions: " << endl;
for (int i = 0; i < prods.size(); i++)
{
   cout << i << ". ";
   printProd(prods[i], -1);
}
fiproc = vector<int>(nterms.size(), 0);
foproc = vector<int>(nterms.size(), 0);
first = vector<set<char>>(nterms.size());
follow = vector<set<char>>(nterms.size());
follow[startSymIdx].insert('$');
for (char c : nterms)
   fillFollow(c);
constructClosures();
lstates.push_back(findClosure(startSymIdx));
for (int i = 0; i < lstates.size(); i++)
{
   for (char c : terms)
```

```
        {
            if (c != '$')
                findGoto(i, c);
        }
        for (char c : nterms)
        {
            if (c != 'Z')
                findGoto(i, c);
        }
    }
    cout << endl
        << "States: " << endl;
    printStates();
    cout << "Gotos: " << endl;
    printGotos();
    cout << endl
        << endl;
    cout << "Follow(): " << endl;
    printFollows();
    table = vector<vector<string>>(
        lstates.size(),
        vector<string>(nterms.size() + terms.size(), "-"));
    fillTable();
    printTable();
    cout << endl
        << "Parsing Input: " << endl;
    parse(input);
}
```

// S->L=R|S->R|L->*R|L->id|R->L

Input and Output:

Input Grammar: (NT = {i , d , = , *}, T = {S, L, R}, P={ S->L=R | S->R | L->*R | L->id | R->L}, S = {S})
Input String: *d=id

```
PS C:\Users\vishw> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-8\" ; if ($?) { g++ SLR_Parser.cpp -o SLR_Parser } ; if ($?) { .\SLR_Parser }
Enter terminals seperated by comma: =,*,i,d
Enter non-terminals seperated by comma: S,L,R
Enter starting symbol: S
Enter productions seperated by |: S->L=R|S->R|L->*R|L->id|R->L
Enter input string: id=id

Augmented productions:
0. S'->S
1. S->L=R
2. S->R
3. L->*R
4. L->id
5. R->L

States:
I0
S'->.S
S->.L=R
L->.*R
L->.id
S->.R
R->.L

I1
L->*.R
R->.L
L->.*R
L->.id

I2
L->i.d

I3
S->L.=R
R->L.

I4
S->R.
```

```
I5
S'->S.

I6
R->L.

I7
L->*R.

I8
L->id.

I9
S->L=.R
R->.L
L->.*R
L->.id

I10
S->L=R.

Gotos:
goto(0,L): 3 | goto(0,R): 4 | goto(0,S): 5 | goto(0,*): 1 | goto(0,=): - | goto(0,d): - | goto(0,i): 2 | goto(1,L): 6 |
goto(1,R): 7 | goto(1,S): - | goto(1,*): 1 | goto(1,=): - | goto(1,d): - | goto(1,i): 2 | goto(2,L): - | goto(2,R): - |
goto(2,S): - | goto(2,*): - | goto(2,=): - | goto(2,d): 8 | goto(2,i): - | goto(3,L): - | goto(3,R): - | goto(3,S): - |
goto(3,*): - | goto(3,=): 9 | goto(3,d): - | goto(3,i): - | goto(4,L): - | goto(4,R): - | goto(4,S): - | goto(4,*): - |
goto(4,=): - | goto(4,d): - | goto(4,i): - | goto(5,L): - | goto(5,R): - | goto(5,S): - | goto(5,*): - | goto(5,=): - |
goto(5,d): - | goto(5,i): - | goto(6,L): - | goto(6,R): - | goto(6,S): - | goto(6,*): - | goto(6,=): - | goto(6,d): - |
goto(6,i): - | goto(7,L): - | goto(7,R): - | goto(7,S): - | goto(7,*): - | goto(7,=): - | goto(7,d): - | goto(7,i): - |
goto(8,L): - | goto(8,R): - | goto(8,S): - | goto(8,*): - | goto(8,=): - | goto(8,d): - | goto(8,i): - | goto(9,L): 6 |
goto(9,R): 10 | goto(9,S): - | goto(9,*): 1 | goto(9,=): - | goto(9,d): - | goto(9,i): 2 | goto(10,L): - | goto(10,R): - |
goto(10,S): - | goto(10,*): - | goto(10,=): - | goto(10,d): - | goto(10,i): - |

Follow():
L: {$ = }
R: {$ = }
S: {$ }
Z: {$ }
```

```
SLR Parsing Table:
STATE   |                    ACTION                        |  GOTO     |
    -   |    $    |    *    |    =    |    d    |    i    | L | R | S |
    0   |    -    |   s1    |    -    |    -    |   s2    | 3 | 4 | 5 |
    1   |    -    |   s1    |    -    |    -    |   s2    | 6 | 7 | - |
    2   |    -    |    -    |    -    |   s8    |    -    | - | - | - |
    3   |   r5    |    -    |   s9    |    -    |    -    | - | - | - |
    4   |   r2    |    -    |    -    |    -    |    -    | - | - | - |
    5   |  ACC    |    -    |    -    |    -    |    -    | - | - | - |
    6   |   r5    |    -    |   r5    |    -    |    -    | - | - | - |
    7   |   r3    |    -    |   r3    |    -    |    -    | - | - | - |
    8   |   r4    |    -    |   r4    |    -    |    -    | - | - | - |
    9   |    -    |   s1    |    -    |    -    |   s2    | 6 |10 | - |
   10   |   r1    |    -    |    -    |    -    |    -    | - | - | - |

Parsing Input:
     STACK         |     INPUT         |             ACTION                |
         0         |    id=id$         |            SHIFT 2                |
       0i2         |     d=id$         |            SHIFT 8                |
     0i2d8         |      =id$         |       REDUCE L->id                |
       0L3         |      =id$         |            SHIFT 9                |
     0L3=9         |       id$         |            SHIFT 2                |
   0L3=9i2         |        d$         |            SHIFT 8                |
 0L3=9i2d8         |         $         |       REDUCE L->id                |
   0L3=9L6         |         $         |        REDUCE R->L                |
  0L3=9R10         |         $         |      REDUCE S->L=R                |
       0S5         |         $         |             ACCEPT                |
PS C:\Users\vishw\Coding\Compiler-Lab\Week-8>
```

Result:

The SLR parser has been successfully written in C++ and executes as expected.